

【科普情景类】

大语言模型漫谈

Q1: 解释这两种描述为什么并不矛盾

预测下一个 token 是大模型背后的训练目的，但像人一样“理解要求并回答问题”是它经过指令训练后的表现形式。如果“理解”指成功识别内容并根据关系产生恰当回答，那大模型在某种意义上已经表现出了这种理解；但如果理解指的是像人一样拥有意识或感受，那就无法证明大模型确实存在这样的心智。所以这关乎我们对“理解”的定义问题。

Q2: 既然统计已有材料，并总结规律就可以让根据输入产生的输出精准，那么我们用圆周率作为训练，训练一个可以吐数字的机器，能不能帮我们解决计算圆周率的问题？

如果只给模型圆周率已经算出的数字，它主要会记忆这些数字，根据给出的数字形成经验，猜测下一位数字，而不是真正地在“计算”圆周率。这是不准确的。因此它不能计算出材料外的新位数。

Q3: 看起来，LLM 生成的文本和人类生活所频繁接触的语料完全不一样，没有人说话会十分关切，或者这样头头是道，即使在人类文献中，叙述风格类似大语言模型的也寥寥无几，可为什么 LLM 生成的文本风格就是那样的不同？

大模型在学习语料的同时，也会接受指令训练。这些训练可能会鼓励大模型回答要清晰有逻辑、语气礼貌、表达关切等等。同时，大模型由于面对众多用户，所以会减少一些“个性鲜明”的表达，选择中性、结构化、不容易出错的回答。

Q4: 能不能让 LLM 输出的语料来自己优化自己？

这种方式有可行性，但存在限制。把输出重新用于训练可以让大模型更稳定地产生正确的结果。但如果输出的语料中存在错误，可能会把错误反复放大，出现更多错误。并且如果仅用 LLM 自己的语料来训练，可能出现内容越来越单一、语言越来越模板化等等问题。对长远的发展来看有弊无利。

会倒着跑的程序

Q1: 逆指令流是 DOS 时代频繁使用的加密方法，为什么这种加密方法现如今逐渐退居二线？

1. 现代的操作系統已经迭代更新，用户的权限不能修改系统内核，无法像文档里展示的那样修改 INT 1 的处理程序来实现我们的目的。这种基于中断器的逆指令流不适配现代系统操作逻辑。
2. 逆指令流只能改变程序运行顺序，而不能做到真正意义上的加密，有自身的局限性，安全性并不强。例如用 jmp 指令时，可以轻易破解；而用中断器的逆指令流在现代系统已经没有可行性。
3. 如 INT 1+HDB 和 INT 3 的指令，它们会大量增加程序运行的步骤，降低效率
4. Dos 实模式由于直接对应物理内存地址，因此有很大风险误修改其他程序，进而导致重要内容的错误。以及一旦错误可能导致整个系统的崩溃。
5. 有更加合理有效的现代加密手段

6. 但似乎逆指令流在严格意义上来说并不是“加密”，如凯撒和栅栏密码这样的古典密码学和现代密码学，都有明文与密文之分，但逆指令流只是让程序不容易被别人读懂？通过搜索发现现代类似逆指令流目的的方法有加壳、控制流平坦化、指令替换等等。
7. 总之逆指令流只在 DOS 时代频繁使用，现今已经由于上述种种原因不再受到人们的青睐。

Q2：既然不能访问真实的物理地址，为什么很多保留底层接口的语言还会存有“指针”这个概念？

现代程序中的指针通常保存的是虚拟地址。指针可以用来保存函数的位置、建立链表、树等数据结构、管理动态申请的内存、传递对象的位置等等，指针表达的是程序地址空间中的位置关系，不必直接对应物理地址。

Q3：现代程序加密方法有哪些，你能否尝试阐述他们是否受实模式的影响？

加壳：把原始程序压缩或加密保存，运行时再恢复。

控制流平坦化：把原来明显的层级结构改造成统一调度结构。类似 `next_ip`

虚拟机混淆：把代码转换成自定义指令，再由程序内置的解释器执行。

字符串与资源加密：加密程序中的文字、图片、配置等，使用时再解密。

动态代码解密：只在代码即将执行时临时将其解密。

反调试：检测调试器，发现后退出或改变程序行为。

完整性检查：检查程序是否被修改，发现篡改后拒绝运行。

以上均不依赖实模式。实模式中程序给出的地址会比较直接地对应物理内存位置，缺少现代系统中的地址隔离和权限保护。它主要影响的是直接修改中断向量表这类接触系统内核的手段。

【心智模型类】

with 漫谈

根据 PEP 343, `with` 使用的是“上下文管理协议”。上下文是一段代码运行时临时建立的环境，而上下文管理器负责进入和退出这个环境。

它可以简化管理为 `try...finally` 和 `__enter__()`、`__exit__()` 的作用。它在正常退出、发生 Python 异常或提前离开代码块时，能够显著降低资源未释放的问题，防止出现由于发生异常或其他原因导致文件没有正常关闭的情况。

【实践实操类】

1. 25 年初曾在本地部署了 deepseek R1 的 7b 版本。

基本全靠 B 站教程，当时搜了一大堆视频，以及下载 docker，使用 `cmd` 之类的。

具体过程已经忘了，因为纯是折腾了一下，而且一秒只能输出三个字。所以安装完成后就再也没用过了 orz。

2. 暑假帮助一位学长完成了发布在 github 上的程序测试，提交了一份 PR。

PS：会熟练使用 codex。在其辅助下做出了自己的 qq 机器人，不断调试，以及在尝试更进一步发展。